Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

KAPITEL

1

Objekte und Klassen

Lernziele

Zentrale Konzepte in diesem Kapitel: Objekte, Methoden, Klassen, Parameter

Zeit loszulegen und mit unserer Einführung in die objektorientierte Programmierung zu beginnen. Um Programmieren zu lernen, bedarf es zweierlei: ein wenig Theorie und viel, viel Praxis. Das eine wie das andere bekommen Sie in diesem Buch geboten, sodass sich beide gegenseitig unterstützen und befruchten können.

Zuerst müssen wir zwei zentrale Konzepte der Objektorientierung verstehen: *Objekte* und *Klassen*. Sie bilden die Basis jedweder objektorientierten Programmierung, gleich in welcher objektorientierten Sprache. Lassen Sie uns also mit einer kurzen Besprechung dieser beiden grundlegenden Konzepte beginnen.

1.1 Objekte und Klassen

Wenn Sie ein Computerprogramm in einer objektorientierten Sprache schreiben, dann erstellen Sie in Ihrem Computer ein Modell eines Ausschnitts der realen Welt. Dieser Ausschnitt setzt sich aus den *Objekten* zusammen, die im Anwendungsbereich vorkommen. Diese Objekte müssen im zu erstellenden Computermodell geeignet repräsentiert werden. Die Objekte des Anwendungsbereichs unterscheiden sich je nach Programm, das Sie schreiben. Es können Wörter und Absätze sein, wenn Sie eine Textverarbeitung programmieren, Nutzer und Nachrichten, wenn Sie an einem sozialen Netzwerk arbeiten, oder Monster, wenn Sie ein Computerspiel schreiben.

Konzept

Java-**Objekte** modellieren Objekte eines Anwendungsbereichs.

Objekte können klassifiziert werden. Eine Klasse beschreibt – auf abstrakte Weise – alle Objekte einer bestimmten Art.

Wir können diese etwas abstrakten Aussagen klarer machen, indem wir ein Beispiel betrachten. Nehmen Sie an, Sie möchten eine Verkehrssimulation entwerfen. Ein Begriff, mit dem Sie dann sicherlich umgehen werden, ist „Auto“. Was ist ein Auto in unserem Kontext? Ist es eine Klasse oder ein Objekt? Einige zusätzliche Fragen können uns helfen, eine Entscheidung zu treffen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Welche Farbe hat ein Auto? Welche Höchstgeschwindigkeit hat es? Wo befindet es sich gerade?

Konzept

Objekte werden durch **Klassen** erzeugt. Eine Klasse beschreibt eine bestimmte Art von Objekten; Objekte repräsentieren individuelle Instanzen einer Klasse.

Wie Sie sicher bemerkt haben, können wir diese Fragen nicht beantworten, bevor wir über ein bestimmtes Auto sprechen. Der Grund ist, dass wir uns mit dem Wort „Auto" auf die *Klasse* Auto beziehen – wir reden über Autos im Allgemeinen, nicht über ein bestimmtes.

Wenn ich aber sage: „Das Auto, das zu Hause in meiner Garage parkt", dann kann ich die obigen Fragen beantworten. Das Auto ist rot, es fährt nicht sehr schnell, und es steht in meiner Garage. Jetzt rede ich über ein Objekt – über ein spezielles Exemplar eines Autos.

Wir nennen üblicherweise ein bestimmtes Objekt eine *Instanz* (*instance,* Exemplar). Diesen Begriff werden wir von nun an recht häufig verwenden. „Instanz" ist ungefähr synonym zu „Objekt" – wir beziehen uns auf Objekte als Instanzen, wenn wir betonen wollen, dass sie von einer bestimmten Klasse sind (wie in „dieses Objekt ist eine Instanz der Klasse `Auto`").

Bevor wir diese etwas theoretische Diskussion fortsetzen, wollen wir zunächst ein Beispiel betrachten.

1.2 Instanzen erzeugen

Starten Sie BlueJ und öffnen Sie das Projekt *Figuren*.[1] Sie sollten daraufhin ein Fenster sehen, das dem in Abbildung 1.1 ähnelt.

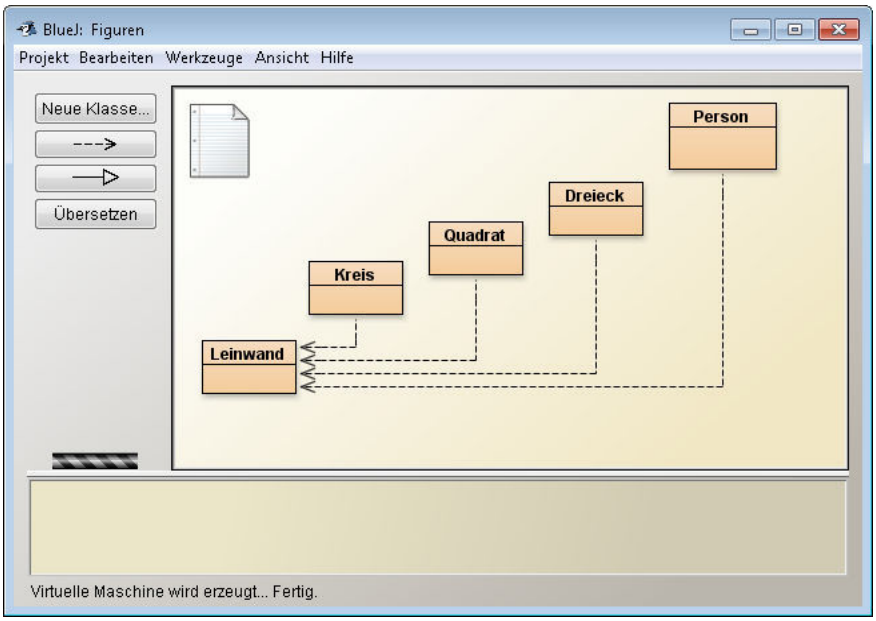


Abbildung 1.1 Das Projekt *Figuren* in BlueJ.

1 Wir empfehlen, dass Sie beim Lesen dieses Buches immer wieder selbst aktiv werden und die Übungen durcharbeiten. An dieser Stelle nehmen wir an, dass Sie bereits wissen, wie man BlueJ startet und das Beispielprojekt öffnet. Falls dies nicht der Fall ist, lesen Sie bitte zuerst Anhang A.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

In diesem Fenster sollte ein Diagramm sichtbar sein. Jedes der gefärbten Rechtecke in diesem Diagramm repräsentiert eine Klasse in diesem Projekt. Die Klassen dieses Projekts heißen **Kreis**, **Quadrat**, **Dreieck**, **Person** und **Leinwand**.

Klicken Sie mit der rechten Maustaste auf die Klasse **Kreis** und wählen Sie

```
new Kreis()
```

aus dem Kontextmenü. Das System fragt Sie nun nach einem „Instanznamen" – klicken Sie auf OK, der vorgegebene Name ist vorerst gut genug. Im unteren Bereich des Fensters sollte nun ein rotes Rechteck erscheinen, das mit „kreis1" beschriftet ist (Abbildung 1.2).

Abbildung 1.2
Eine Instanz in der Objektleiste.

Gerade haben Sie Ihr erstes Objekt erzeugt! „Kreis", das rechteckige Symbol in Abbildung 1.1, repräsentiert die Klasse **Kreis**; **kreis1** ist eine frisch erzeugte Instanz dieser Klasse. Den Bereich am unteren Ende des Fensters, in dem die Instanz angezeigt wird, nennen wir die *Objektleiste*.

Konvention

Wir beginnen den Namen einer Klasse mit einem Großbuchstaben (wie in **Kreis**) und benennen Objekte mit Kleinbuchstaben (wie in **kreis1**). Dies hilft uns, die Dinge voneinander zu unterscheiden.

Übung 1.1 Erzeugen Sie einen weiteren Kreis. Erzeugen Sie dann ein Quadrat.

1.3 Methoden aufrufen

Führen Sie einen Rechtsklick auf einer der Kreis-Instanzen aus (nicht auf der Klasse!). Es erscheint ein Kontextmenü (Abbildung 1.3) mit mehreren Einträgen. Wählen Sie **sichtbarMachen** aus – dies bewirkt, dass eine Repräsentation dieses Kreises in einem eigenen Fenster angezeigt wird (Abbildung 1.4).

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

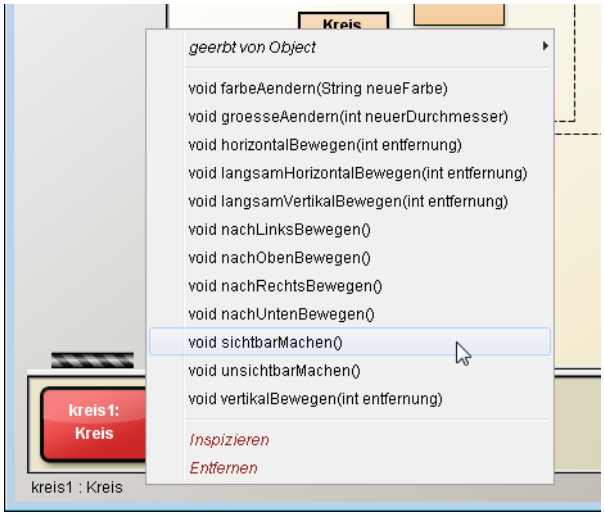


Abbildung 1.3 Das Kontextmenü eines Objekts mit der Liste seiner Operationen.

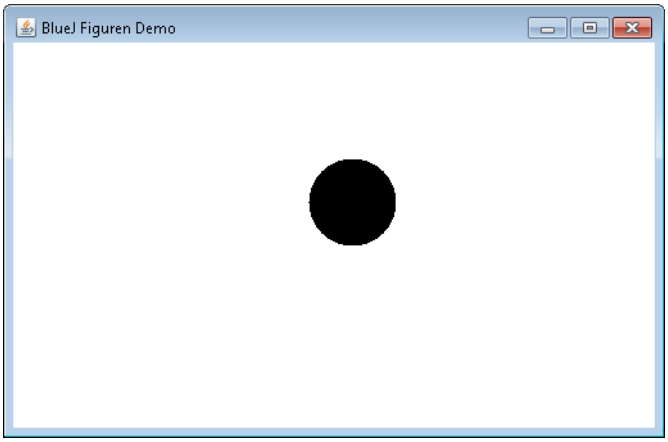


Abbildung 1.4 Die Visualisierung eines Kreises.

Konzept

Wir können mit Objekten kommunizieren, indem wir ihre **Methoden** aufrufen. Ein Objekt tut üblicherweise etwas, wenn eine seiner Methoden aufgerufen wird.

Sie sehen einige weitere Einträge im Kontextmenü des Kreises. Versuchen Sie, `nachRechtsBewegen` oder `nachUntenBewegen` auszuwählen und damit den Kreis näher an die Ränder des Fensters zu rücken. Probieren Sie auch `unsichtbarMachen` und `sichtbarMachen`, um den Kreis zu verstecken und wieder anzuzeigen.

Übung 1.2 Was geschieht, wenn Sie `nachUntenBewegen` zweimal aufrufen? Oder dreimal? Was passiert, wenn Sie `unsichtbarMachen` zweimal aufrufen?

Die Einträge im Kontextmenü der Kreis-Instanz repräsentieren Operationen, mit denen die Instanz manipuliert werden kann. Diese Operationen nennen wir Methoden. Die übliche Sprechweise ist, dass diese Methoden aufgerufen werden. Wir wollen diese Terminologie von nun an einhalten. Wir können Sie beispielsweise auffordern, „die Methode `nachRechtsBewegen` an dem Objekt `kreis1`“ aufzurufen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

1.4 Parameter

Rufen Sie nun die Methode **horizontalBewegen** auf. Es erscheint ein Dialog, der Sie um eine Eingabe bittet (Abbildung 1.5). Tippen Sie „50“ ein und klicken Sie auf OK. Der Kreis wird sich daraufhin um 50 Bildschirmpunkte nach rechts bewegen.[2]

Konzept

Methoden können **Parameter** haben, mit denen zusätzliche Informationen für eine Aufgabe angegeben werden.

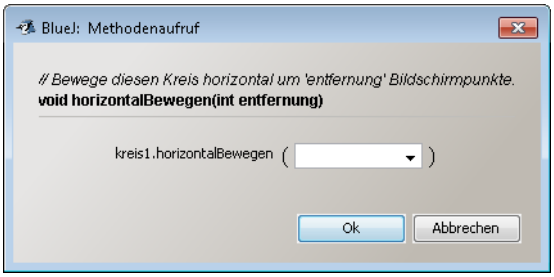


Abbildung 1.5
Ein Dialog zum Aufruf von Methoden.

Die Methode **horizontalBewegen**, die Sie gerade aufgerufen haben, ist so gestaltet, dass sie zusätzliche Informationen von Ihnen benötigt, um ihre Aufgabe erfüllen zu können. In diesem Fall ist diese zusätzliche Information, um wie viele Bildschirmpunkte der Kreis bewegt werden soll. Die Methode **horizontalBewegen** ist somit flexibler als die Methoden **nachRechtsBewegen** und **nachLinksBewegen**. Die Letzteren bewegen den Kreis immer um eine festgelegte Entfernung, wohingegen **horizontalBewegen** die Möglichkeit bietet anzugeben, wie weit der Kreis bewegt werden soll.

Übung 1.3 Bevor Sie weiterlesen, sollten Sie die Methoden **vertikalBewegen**, **langsamVertikalBewegen** und **groesseAendern** aufrufen. Finden Sie heraus, wie man **horizontalBewegen** benutzen kann, um den Kreis 70 Bildschirmpunkte nach links zu bewegen.

Die zusätzlichen Werte, die manche Methoden benötigen, werden *Parameter* genannt. Eine Methode gibt an, welche Art von Parametern sie erwartet. Wenn beispielsweise die Methode **horizontalBewegen** wie in Abbildung 1.5 aufgerufen wird, dann wird im Dialogfenster die Zeile

```
void horizontalBewegen(int entfernung)
```

im oberen Bereich angezeigt. Dies ist der *Kopf* der Methode. Der Kopf liefert Informationen über eine Methode. Der Teil zwischen den Klammern (**int entfernung**) liefert Informationen über benötigte Parameter. Für jeden Parameter wird ein Typ und ein Name definiert. Der Kopf im obigen Beispiel sagt aus, dass die Methode **horizontalBewegen** einen Parameter vom Typ **int** mit dem Namen **entfernung** erwartet. Der Name gibt dabei einen Hinweis auf die Bedeutung des Parameters. Der Name einer Methode und die Parametertypen, die im Kopf vorgefunden werden, werden zusammen als die *Signatur* der Methode bezeichnet.

Konzept

Der Name und die Parametertypen einer Methode werden als ihre **Signatur** bezeichnet. Sie benennen die benötigten Informationen für einen Aufruf der Methode.

2 Bildschirmpunkte werden auch als Pixel bezeichnet. Der Bildschirm besteht aus einem Raster von Bildschirmpunkten.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

1.5 Datentypen

Konzept

Parameter haben **Typen**. Ein Typ definiert, welche Arten von Werten ein Parameter annehmen kann.

Ein Typ gibt an, welche Art von Information als Parameter übergeben werden kann. Der Typ `int` beispielsweise bezeichnet ganze Zahlen (aus dem Englischen *integer*, daher die Abkürzung „int“).

Im obigen Beispiel besagt die Signatur von `horizontalBewegen`, dass wir, bevor die Methode ausgeführt werden kann, eine ganze Zahl festlegen müssen, die die Entfernung angibt. Das Eingabefeld in Abbildung 1.5 ermöglicht diese Eingabe.

In den bisherigen Beispielen haben wir nur den Datentyp `int` kennengelernt. Die Parameter der Methoden zum Bewegen und der Methode `groesseAendern` sind alle von diesem Typ.

Eine nähere Betrachtung des Kontextmenüs einer Instanz zeigt, dass die Einträge in diesem Menü bereits die Parameterinformation enthalten. Wenn eine Methode keine Parameter hat, folgen auf den Namen nur leere Klammern. Wenn sie einen Parameter hat, wird der Typ dieses Parameters angezeigt. In der Liste der Methoden für einen Kreis können Sie eine Methode mit einem anderen Parametertyp entdecken: Die Methode `farbeAendern` hat einen Parameter vom Typ `String`.

Der Typ `String` signalisiert, dass ein Stück Text (beispielsweise ein Wort oder ein Satz) erwartet wird. Da ein solcher Text sich aus einzelnen Zeichen zusammensetzt, bezeichnen wir ihn als eine Zeichenkette. Solche Zeichenketten werden in doppelten Anführungsstrichen angegeben. Um beispielsweise das Wort *rot* als Zeichenkette zu übergeben, tippen Sie:

```
"rot"
```

Der Aufrufdialog enthält außerdem über der Zeile mit dem Methodenkopf einen Textabschnitt, den wir als Kommentar bezeichnen. *Kommentare* werden angegeben, um dem menschlichen Leser Informationen zu geben; sie werden ausführlicher in Kapitel 2 behandelt. Der Kommentar der Methode `farbeAendern` beschreibt beispielsweise, welche Farbnamen dem System bekannt sind.

Übung 1.4 Rufen Sie die Methode `farbeAendern` an einer Ihrer Kreis-Instanzen auf und übergeben Sie die Zeichenkette `"rot"`. Dies sollte die Farbe des Kreises ändern. Versuchen Sie es auch mit anderen Farben.

Übung 1.5 Dies ist ein sehr einfaches Beispiel, es werden nicht sehr viele Farben unterstützt. Was passiert, wenn Sie eine Farbe angeben, die nicht unterstützt wird?

Übung 1.6 Rufen Sie die Methode `farbeAendern` auf und geben Sie die Farbe ohne die Anführungszeichen an. Was passiert?

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Fallstrick

Ein typischer Fehler von Programmieranfängern ist, dass sie bei Angabe eines Parameters vom Typ `String` die doppelten Anführungszeichen vergessen. Wenn Sie beispielsweise `blau` statt `"blau"` tippen, bekommen Sie eine Fehlermeldung in der Art: "Error: cannot resolve symbol – variable blau".

Java unterstützt etliche weitere Datentypen, einschließlich Typen für Fließkommazahlen und einzelne Zeichen. Wir werden diese hier nicht alle diskutieren, sondern zu einem späteren Zeitpunkt auf sie zurückkommen. Wenn Sie jetzt mehr darüber wissen wollen, dann können Sie sich Anhang B ansehen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

1.6 Eine Klasse, viele Instanzen

Sobald Sie eine Klasse haben, können Sie beliebig viele Instanzen dieser Klasse erzeugen. Mit der Klasse `Kreis` können Sie beliebig viele Kreise erzeugen, mit der Klasse `Quadrat` beliebig viele Quadrate.

Übung 1.7 Erzeugen Sie mehrere Kreis-Instanzen auf der Objektleiste, indem Sie mehrfach den Eintrag `new Kreis()` aus dem Kontextmenü der Klasse `Kreis` auswählen. Machen Sie die Instanzen sichtbar und bewegen Sie sie mit den Bewegungsmethoden. Machen Sie einen Kreis groß und gelb, einen anderen klein und grün. Probieren Sie auch die anderen Figuren: Erzeugen Sie ein paar Dreiecke und Quadrate. Ändern Sie deren Positionen, Größen und Farben.

Jedes dieser Objekte hat seine eigene Position, Farbe und Größe. Sie ändern eine Eigenschaft eines Objekts (etwa seine Farbe), indem Sie eine Methode an dem Objekt aufrufen. Dieser Aufruf beeinflusst dieses spezielle Objekt, aber keine anderen Objekte.

Konzept

Eine Klasse, **viele Instanzen:** Von einer Klasse können viele gleichartige Instanzen erzeugt werden.

Sie haben möglicherweise auch ein weiteres Detail über Parameter entdeckt. Sehen Sie sich die Methode `groesseAendern` an einem Dreieck an. Der Methodenkopf lautet:

```
void groesseAendern (int neueHoehe, int neueBreite)
```

Dies ist ein Beispiel für eine Methode mit mehr als einem Parameter. Diese Methode hat zwei Parameter, die im Kopf durch ein Komma voneinander getrennt sind. Allgemein kann eine Methode beliebig viele Parameter haben.

1.7 Zustand

Konzept

Objekte haben einen **Zustand**. Dieser Zustand wird durch Werte repräsentiert, die in Datenfeldern gehalten werden.

Die Menge der momentanen Werte der Attribute, die ein Objekt definieren (wie *x*- und *y*-Position, Farbe, Durchmesser und Sichtbarkeit eines Kreises), wird auch als der *Zustand* des Objekts bezeichnet. Dies ist ein weiteres Beispiel der allgemein üblichen Terminologie, an die wir uns von nun an halten werden.

In BlueJ kann der Zustand eines Objekts genauer untersucht werden, indem der Eintrag INSPIZIEREN aus dem Kontextmenü einer Instanz ausgewählt wird. Wenn ein Objekt auf diese Weise untersucht wird, erscheint ein Fenster, das dem in Abbildung 1.6 ähnlich ist. Dieses Fenster nennen wir den *Objektinspektor*.

Abbildung 1.6 Ein Objektinspektor, der Details eines Objekts anzeigt.

Übung 1.8 Sorgen Sie dafür, dass mehrere Objekte auf der Objektleiste liegen, und untersuchen Sie diese der Reihe nach. Versuchen Sie, den Zustand eines Objekts zu ändern (beispielsweise durch den Aufruf von `nachLinksBewegen`), während der Objektinspektor geöffnet ist. Sie sollten beobachten können, wie sich die Werte im Objektinspektor ändern.

Einige Methoden verändern den Zustand eines Objekts, wenn sie aufgerufen werden. Die Methode `nachLinksBewegen` ändert beispielsweise das Attribut `xPosition`. In Java werden diese Attribute von Objekten häufig *Datenfelder* oder einfach nur Felder (*fields*) genannt.

1.8 Das Innenleben eines Objekts

Beim Untersuchen verschiedener Objekte stellen wir fest, dass alle Instanzen *derselben* Klasse auch die gleichen Datenfelder haben. Das heißt, Anzahl, Typen und Namen der Datenfelder sind gleich, während der aktuelle Wert eines Datenfelds in jedem Objekt anders sein kann. Im Unterschied dazu haben Instanzen *unterschiedlicher* Klassen auch unterschiedliche Datenfelder. Ein Kreis beispielsweise hat ein Datenfeld „durchmesser", während ein Dreieck die Datenfelder „breite" und „hoehe" hat.

Dies ist deshalb so, weil Anzahl, Typen und Namen von Datenfeldern durch die Klasse definiert sind, nicht in einem Objekt. Die Klasse `Kreis` definiert, dass jede

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Kreis-Instanz fünf Datenfelder hat, und zwar **durchmesser**, **xPosition**, **yPosition**, **farbe** und **istSichtbar**. Sie legt auch fest, welche Typen diese Datenfelder haben. Die ersten drei sind vom Typ **int**, die Farbe ist vom Typ **String** und **istSichtbar** ist vom Typ **boolean**. (Eine boolesche Variable kann einen von zwei Werten annehmen: **true** und **false**. Der Typ **boolean** wird später noch ausführlicher diskutiert.)

Wenn eine Instanz der Klasse **Kreis** erzeugt wird, dann hat diese Instanz automatisch diese Datenfelder. Die Werte der Datenfelder werden in der Instanz gespeichert. Dies garantiert, dass jede Instanz beispielsweise eine Farbe hat, ermöglicht aber auch, dass jede Instanz eine andere Farbe haben kann (Abbildung 1.7).

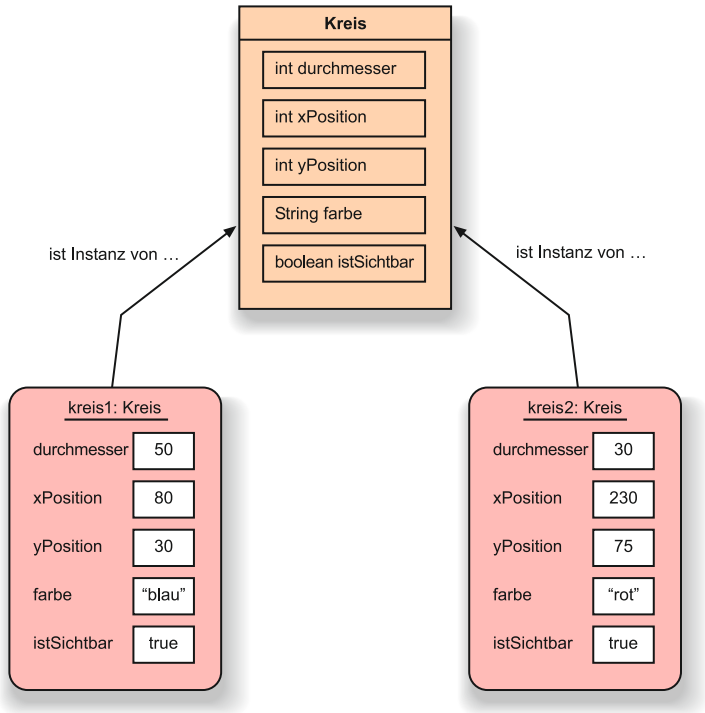


Abbildung 1.7
Eine Klasse und ihre Instanzen mit Datenfeldern und Werten.

Das Gleiche gilt für Methoden. Die Methoden werden in der Klasse eines Objekts definiert. Folglich haben alle Instanzen einer gegebenen Klasse die gleichen Methoden. Aber die Methoden werden an konkreten Instanzen aufgerufen. Auf diese Weise ist eindeutig, welches Objekt seinen Zustand ändern muss, wenn beispielsweise die Methode **nachRechtsBewegen** aufgerufen wird.

Übung 1.9 In Abbildung 1.8 auf Seite 42 sehen Sie zwei Bilder. Suchen Sie sich eines dieser Bilder aus und versuchen Sie, es mithilfe der Figuren aus dem Projekt *Figuren* nachzustellen. Notieren Sie gleichzeitig, was Sie im Einzelnen tun müssen, um dies zu erreichen. Könnten Sie es auch auf andere Weise erreichen?

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

1.9 Java-Code

Wenn wir in Java programmieren, schreiben wir eigentlich Befehle nieder, um Methoden an Objekten aufzurufen – so wie wir es oben für unsere Figur-Objekte gemacht haben. Allerdings tun wir dies nicht interaktiv, indem wir mit der Maus Methoden aus einem Menü auswählen, sondern indem wir die Befehle als Text eintippen. In der BlueJ-Konsole können wir uns ansehen, wie diese Befehle als Text aussehen.

Übung 1.10 Wählen Sie im Menü ANSICHT den Befehl KONSOLE ANZEIGEN. Damit wird ein anderes Fenster aufgerufen, das BlueJ zur Ausgabe von Text verwendet. Wählen Sie in diesem Terminalfenster den Befehl METHODENAUFRUFE PROTOKOLLIEREN im Menü OPTIONEN. Mit dieser Funktion werden unsere Methodenaufrufe (als Text) in der Konsole angezeigt. Erzeugen Sie jetzt einige Objekte, rufen Sie einige ihrer Methoden auf und beobachten Sie die Ausgabe im Konsolenfenster.

Mithilfe des Befehls METHODENAUFRUFE PROTOKOLLIEREN können wir erkennen, dass die Folge, ein Personenobjekt zu erzeugen und seine Methoden **sichtbarMachen** und **nachRechtsBewegen** aufzurufen, als Java-Code in Textform wie folgt aussieht:

```
Person person1 = new Person();
person1.sichtbarMachen();
person1.nachRechtsBewegen();
```

An diesem Text lassen sich einige Dinge beobachten:

- Wir sehen, wie der Code aussieht, wenn ein Objekt erzeugt und mit einen Namen verbunden wird. Technisch gesehen *speichern wir das Person-Objekt in einer Variablen*. Was dies genau bedeutet, besprechen wir ausführlich im nächsten Kapitel.
- Wir sehen, dass wir für den Aufruf einer Methode an einem Objekt den Namen des Objekts gefolgt von einem Punkt und dem Namen der Methode schreiben. Der Befehl endet mit einer Parameterliste beziehungsweise einem leeren Paar Klammern, wenn es keine Parameter gibt.
- Alle Java-Anweisungen enden mit einem Semikolon.

Doch wir wollen nicht nur Java-Anweisungen betrachten, sondern auch welche eingeben. Hierzu verwenden wir die *Direkteingabe*. (Sie können jetzt die Funktion METHODENAUFRUFE PROTOKOLLIEREN ausschalten und die Konsole schließen.)

Übung 1.11 Wählen Sie im Menü ANSICHT den Befehl DIREKTEINGABE ANZEIGEN aus. Es sollte ein neues Fenster neben der Objektleiste in Ihrem BlueJ-Hauptfenster erscheinen, bei dem es sich um die *Direkteingabe* handelt. Hier können Sie Ihren Java-Code eingeben.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

In der Direkteingabe können Sie Java-Code eingeben, der das Gleiche macht wie der Code, den wir zuvor interaktiv erstellt haben.

Übung 1.12 Geben Sie in der Direkteingabe den obigen Code ein, um ein `Person`-Objekt zu erzeugen und ihre Methoden `sichtbarMachen` und `nachRechtsBewegen` aufzurufen.

Das Eintippen dieser Befehle sollte die gleiche Wirkung haben wie der Aufruf des gleichen Befehls aus dem Menü des Objekts. Wenn Sie stattdessen eine Fehlermeldung erhalten, dann haben Sie beim Befehl einen Tippfehler gemacht. Achten Sie also auf Ihre Schreibweise. Sie werden feststellen, dass auch nur ein einziges falsches Zeichen reicht, damit der Befehl nicht funktioniert

Tipp

Sie können in der Direkteingabe unter Verwendung des Aufwärts-Pfeils bereits verwendete Befehle erneut aufrufen.

1.10 Objektinteraktion

Für den nächsten Abschnitt werden wir mit einem anderen Projekt arbeiten. Schließen Sie das Projekt *Figuren*, falls Sie es noch geöffnet haben, und öffnen Sie das Projekt *Haus*.

Übung 1.13 Öffnen Sie das Projekt *Haus*. Erzeugen Sie eine Instanz der Klasse `Bild` und rufen Sie an ihr die Methode `zeichne` auf. Probieren Sie dann die Methoden `inSchwarzWeissAendern` und `inFarbeAendern` aus.

Übung 1.14 Wie erzeugt Ihrer Meinung nach die Klasse `Bild` die Zeichnung?

Fünf der Klassen in diesem Projekt sind identisch mit den Klassen aus dem Projekt *Figuren*. Hier haben wir eine zusätzliche Klasse: `Bild`. Diese Klasse ist so programmiert, dass sie automatisch das tut, was Sie manuell in ***Übung 1.9*** getan haben.

In der Praxis würden wir, wenn wir eine Folge von Aufgaben erledigt haben möchten, dies nicht von Hand tun. Stattdessen würden wir eine Klasse erzeugen, die die Arbeit für uns erledigt. Das genau macht die Klasse `Bild`.

Die Klasse `Bild` ist so geschrieben, dass durch den Aufruf der Methode `zeichne` zwei Quadrat-Instanzen (eines für die Wand, eines für das Fenster) sowie ein Dreieck und ein Kreis erzeugt werden. Wenn anschließend die Methode `zeichne` aufgerufen wird, dann werden diese Instanzen so lange bewegt und in Farbe und Form verändert, bis die Figuren so aussehen, wie in Abbildung 1.8 dargestellt.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

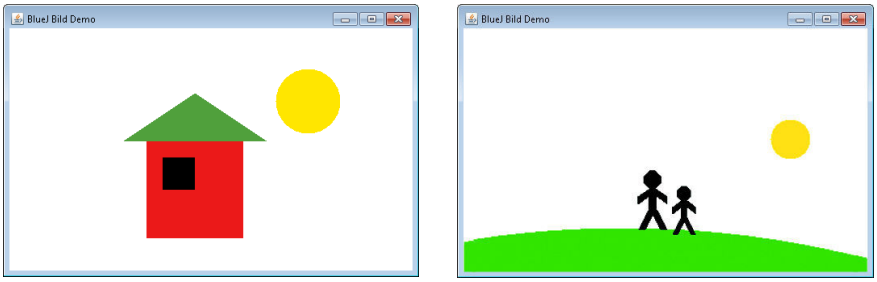


Abbildung 1.8 Zwei Zeichnungen, die mit einigen Figuren erstellt wurden.

Konzept

Methodenaufrufe: Objekte können miteinander kommunizieren, indem sie gegenseitig ihre **Methoden aufrufen**.

Die entscheidenden Punkte an dieser Stelle sind: Objekte können andere Objekte erzeugen und sie können die Methoden dieser Objekte aufrufen. In einem normalen Java-Programm können es sehr schnell Hunderte oder Tausende von Objekten werden. Der Benutzer eines Programms startet es lediglich (indem üblicherweise ein erstes Objekt erzeugt wird), und alle weiteren Objekte werden direkt oder indirekt durch dieses Objekt erzeugt.

Die große Frage ist nun: Wie schreiben wir die Klasse für solch ein Objekt?

1.11 Quelltext

Konzept

Der **Quelltext** einer Klasse legt die Struktur und das Verhalten (die Datenfelder und die Methoden) aller Instanzen dieser Klasse fest.

Jeder Klasse ist ein *Quelltext* zugeordnet. Der Quelltext ist ein Text, der die Details einer Klasse beschreibt. In BlueJ kann der Quelltext einer Klasse betrachtet werden, indem aus dem Kontextmenü der Klasse der Eintrag BEARBEITEN aufgerufen oder ein Doppelklick auf dem Klassensymbol ausgeführt wird.

Übung 1.15 Aktivieren Sie das Kontextmenü der Klasse `Bild`. Sie sollten einen Eintrag BEARBEITEN sehen – wählen Sie diesen aus. Dies startet einen Texteditor, der den Quelltext der Klasse anzeigt.

Der Quelltext ist in der Programmiersprache Java formuliert. Er definiert, welche Datenfelder und Methoden eine Klasse hat, und legt exakt fest, was passiert, wenn eine Methode aufgerufen wird. Im nächsten Kapitel werden wir ausführlich darauf eingehen, was der Quelltext einer Klasse umfasst und wie er strukturiert ist.

Um die Kunst des Programmierens zu erlernen, muss man in erster Linie lernen, solche Definitionen von Klassen zu erstellen. Wir verwenden zu diesem Zweck die Programmiersprache Java (obwohl es viele andere Programmiersprachen gibt, in denen Programme geschrieben werden können).

Wenn Sie eine Änderung am Quelltext vornehmen und den Editor schließen,[3] erscheint das Symbol für die Klasse im Diagramm gestreift. Die Streifen zeigen an, dass der Quelltext geändert wurde. Die Klasse muss nun übersetzt werden, indem der ÜBERSETZEN-Knopf gedrückt wird (falls Sie mehr darüber erfahren möchten, was

3 In BlueJ ist es nicht notwendig, vor dem Schließen des Editors den Text explizit zu speichern. Beim Schließen wird der Quelltext automatisch gespeichert.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

beim Übersetzen passiert, dann lesen Sie den Abschnitt **Programmübersetzung**). Sobald die Klasse übersetzt ist, können Instanzen von ihr erzeugt werden, um an ihnen die Änderungen auszuprobieren.

Programmübersetzung

Wenn Menschen Computerprogramme schreiben, benutzen sie typischerweise eine „höhere" Programmiersprache wie beispielsweise Java. Ein Problem dabei ist, dass ein Computer den Quelltext von Java nicht direkt ausführen kann. Java wurde entworfen, um einigermaßen lesbar für Menschen zu sein, nicht für Computer. Computer arbeiten intern mit einer binären Repräsentation eines Maschinencodes, der sich von Java erheblich unterscheidet. Das Problem ist nun, dass für uns der Maschinencode so unleserlich ist, dass wir ihn nicht direkt schreiben wollen. Wir schreiben lieber in Java. Was ist also zu tun?

Die Lösung ist ein spezielles Programm, der Compiler. Ein Compiler übersetzt Java-Text in Maschinencode. Wir können Java schreiben, den Compiler starten – dieser generiert den Maschinencode – und der Computer kann den Maschinencode ausführen. Als Konsequenz ergibt sich, dass wir jedes Mal, wenn wir den Quelltext ändern, zuerst den Compiler laufen lassen müssen, bevor wir die Klasse zum Erzeugen einer Instanz benutzen können. Andernfalls existiert nicht die Version des Maschinencodes, die der Computer braucht.

Übung 1.16 Finden Sie im Quelltext der Klasse `Bild` den Abschnitt, der das eigentliche Zeichnen ausführt. Ändern Sie ihn so, dass die Sonne blau statt gelb ist.

Übung 1.17 Fügen Sie der Zeichnung eine zweite Sonne hinzu. Dazu sollten Sie sich die Definitionen der Datenfelder zu Beginn der Klassendefinition ansehen. Sie sehen folgendermaßen aus:

```
private Quadrat wand;
private Quadrat fenster;
private Dreieck dach;
private Kreis sonne;
```

An dieser Stelle kann eine Zeile für die zweite Sonne eingefügt werden. Beispielsweise:

```
private Kreis sonne2;
```

Schreiben Sie dann auch die entsprechenden Anweisungen für die zweite Sonne an zwei unterschiedlichen Stellen und machen Sie diese sichtbar, wenn das Bild gemalt wird.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Übung 1.18 *Zusatzaufgabe*. (Dies heißt, dass diese Übung eventuell nicht schnell zu lösen ist. Wir erwarten nicht, dass jeder diese Aufgabe zu diesem Zeitpunkt lösen kann. Wenn Sie es schaffen – großartig. Wenn nicht, dann ist das kein Problem. Die Dinge werden klarer werden, sobald Sie weitergelesen haben. Kommen Sie später auf diese Übung zurück.) Fügen Sie der Version von `Bild` mit nur einer Sonne einen Sonnenuntergang hinzu. Also: Lassen Sie die Sonne langsam untergehen. *Hinweis*: Die Klasse `Kreis` hat eine Methode `langsamVertikalBewegen`, die Sie zu diesem Zweck verwenden können.

Übung 1.19 *Zusatzaufgabe*. Wenn Sie den Sonnenuntergang am Ende der Methode `zeichne` eingefügt haben (sodass die Sonne automatisch untergeht, sobald das Bild gezeichnet ist), so ändern Sie dies nun. Für den Sonnenuntergang soll eine eigene Methode `sonnenuntergang` definiert werden. Auf diese Weise können wir zuerst die Methode `zeichne` aufrufen, um die Zeichnung mit der Sonne zu sehen, und anschließend `sonnenuntergang`, um die Sonne untergehen zu lassen.

Übung 1.20 *Zusatzaufgabe*. Erzeugen Sie eine Person, die nach dem Sonnenuntergang auf das Haus zugeht.

1.12 Ein weiteres Beispiel

In diesem Kapitel haben wir bereits etliche neue Konzepte diskutiert. Um das Verständnis dieser Konzepte zu vertiefen, wollen wir sie noch einmal aus einem anderen Zusammenhang heraus betrachten. Dazu verwenden wir ein anderes Beispiel. Schließen Sie das Projekt *Haus*, falls Sie es noch geöffnet haben, und öffnen Sie das Projekt *Laborkurse*.

Dieses Projekt ist ein vereinfachter Teil einer Studentendatenbank, die entworfen wurde, um Studenten in Laborkursen zu verwalten und Kurslisten auszudrucken.

Übung 1.21 Erzeugen Sie eine Instanz der Klasse `Student`. Sie werden feststellen, dass Sie nicht nur nach dem Namen für die Instanz gefragt werden, sondern auch nach weiteren Parametern. Füllen Sie die Felder aus und klicken Sie auf OK. (Zur Erinnerung: Parameter vom Typ `String` müssen mit doppelten Anführungsstrichen angegeben werden.)

1.13 Aufrufergebnisse

Wie schon zuvor können Sie mehrere Instanzen erzeugen. Und wie zuvor haben die Instanzen Methoden, die jeweils aus ihrem Kontextmenü heraus aufgerufen werden können.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Übung 1.22 Erzeugen Sie einige Instanzen der Klasse **Student**. Rufen Sie die Methode **gibName** an jeder Instanz auf. Erklären Sie, was passiert.

Wenn wir die Methode **gibName** aufrufen, bemerken wir etwas Neues: Methoden können ein Ergebnis liefern. Tatsächlich sagt uns der Kopf einer Methode, ob diese ein Ergebnis liefert oder nicht und von welchem Typ das Ergebnis ist. Der Kopf von **gibName** (dargestellt im Kontextmenü einer Instanz) ist definiert als

```
String gibName()
```

Das Wort **String** vor dem Namen der Methode spezifiziert den Ergebnistyp. In diesem Fall wird ausgesagt, dass der Aufruf dieser Methode ein Ergebnis vom Typ **String** liefert. Die Signatur von **nameAendern** lautet:

```
void nameAendern(String neuerName)
```

Das Schlüsselwort **void** sagt an dieser Stelle, dass diese Methode kein Ergebnis liefert.

Methoden mit einem Ergebnistyp ermöglichen es uns, durch ihren Aufruf Informationen von einem Objekt abzufragen. Dies bedeutet, wir können Methoden entweder benutzen, um den Zustand eines Objekts zu ändern oder um etwas über seinen Zustand herauszufinden. Der Ergebnistyp einer Methode ist nicht Teil ihrer Signatur.

Konzept

Ergebnis: Methoden können Informationen über ein Objekt durch einen **Ergebniswert** zurückliefern.

1.14 Objekte als Parameter

Übung 1.23 Erzeugen Sie eine Instanz der Klasse **Laborkurs**. Die Signatur besagt dabei, dass Sie eine Maximalzahl für die Studenten in dem Kurs angeben müssen (eine ganze Zahl).

Übung 1.24 Rufen Sie die Methode **anzahlStudenten** an dem Laborkurs auf. Was tut sie?

Übung 1.25 Betrachten Sie die Signatur der Methode **trageStudentEin**. Sie werden bemerken, dass der Typ des erwarteten Parameters **Student** ist. Sorgen Sie dafür, dass zwei oder drei Instanzen der Klasse **Student** und eine Instanz von **Laborkurs** auf der Objektleiste liegen. Rufen Sie dann die Methode **trageStudentEin** an der **Laborkurs**-Instanz auf. Mit dem Eingabefokus im Eingabefeld klicken Sie nun auf eine der Instanzen von **Student** – dies trägt den Namen der Instanz in das Eingabefeld ein (Abbildung 1.9). Klicken Sie auf OK und der Student ist in den Laborkurs eingetragen. Tragen Sie weitere Studenten auf diese Weise ein.

Übung 1.26 Rufen Sie die Methode **listeAusgeben** an dem **Laborkurs**-Objekt auf. Es erscheint eine Liste aller Kursteilnehmer auf der BlueJ-Konsole (Abbildung 1.10).

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Abbildung 1.9
Einen Studenten in einen Laborkurs eintragen.

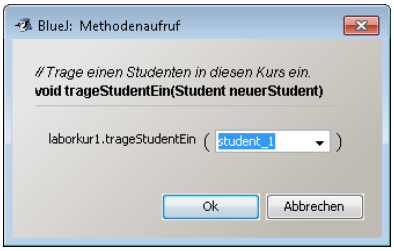
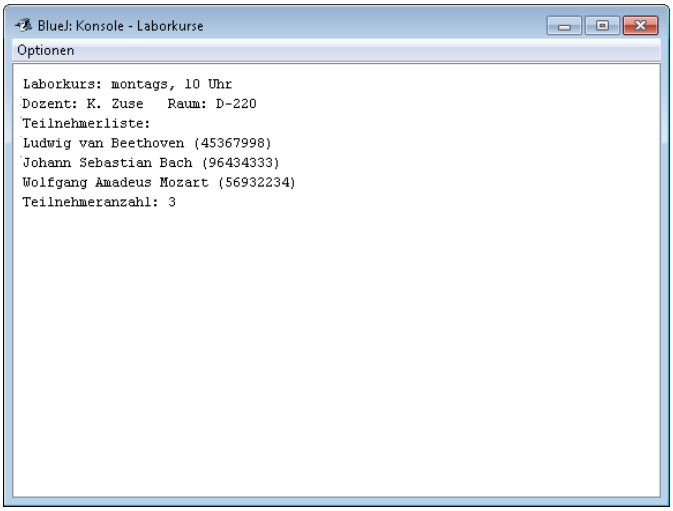


Abbildung 1.10
Ausgabe der Teilnehmerliste einer `Laborkurs`-Instanz.



Diese Übung zeigt, dass Objekte als Parameter an andere Objekte übergeben werden können. Wenn eine Methode ein Objekt als Parameter erwartet, dann wird der Name der Klasse des Objekts als Parametertyp in der Signatur angegeben.

Sehen Sie sich dieses Projekt etwas genauer an. Versuchen Sie, die Konzepte aus dem Projekt *Figuren* in diesem Zusammenhang wiederzuerkennen.

Übung 1.27 Erzeugen Sie drei Studenten mit den folgenden Eigenschaften:

Schneewittchen, Matrikelnummer: 100234, Scheine: 4

Lisa Simpson, Matrikelnummer: 122044, Scheine: 9

Charlie Brown, Matrikelnummer 12003P, Scheine: 1

Tragen Sie die drei anschließend in einen Laborkurs ein und lassen Sie die Kursliste ausgeben.

Übung 1.28 Verwenden Sie den Objektinspektor an dem `Laborkurs`, um herauszufinden, welche Datenfelder er hat.

Übung 1.29 Setzen Sie für einen Laborkurs den Dozenten, den Raum und die Anfangszeit fest. Überprüfen Sie anschließend in der Kursliste in der Konsole, ob diese Informationen angezeigt werden.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Zusammenfassung

In diesem Kapitel haben wir die Grundlagen von Klassen und Objekten untersucht. Wir haben gesehen, dass Objekte durch Klassen definiert werden. Klassen repräsentieren das generelle Konzept eines Gegenstands, während Objekte konkrete Instanzen einer Klasse repräsentieren. Wir können von jeder Klasse beliebig viele Instanzen erzeugen.

Objekte haben Methoden, damit wir mit ihnen kommunizieren können. Wir können eine Methode benutzen, um den Zustand eines Objekts zu ändern oder um Informationen über ein Objekt zu bekommen. Methoden können Parameter haben, und Parameter haben Typen. Methoden definieren einen Ergebnistyp, der festlegt, was sie zurückliefern. Wenn der Ergebnistyp `void` ist, dann liefern sie nichts zurück.

Objekte speichern Daten in ihren Datenfeldern (die auch Typen haben). Die Werte, die in den Datenfeldern eines Objekts gehalten werden, definieren seinen Zustand.

Objekte werden auf der Basis von Klassendefinitionen erzeugt, die in einer bestimmten Programmiersprache formuliert sind. Der überwiegende Anteil der Programmierung in Java beschäftigt sich mit dem Erstellen von Klassendefinitionen. Ein großes Java-Programm besteht aus vielen Klassen, jede mit vielen Methoden, die sich gegenseitig auf vielfältige Weise aufrufen.

Um Java-Programme schreiben zu können, müssen wir lernen, wie man Klassendefinitionen mit Datenfeldern und Methoden schreibt und wie man diese Klassen geeignet miteinander kombiniert. Der Rest dieses Buches wird sich genau damit beschäftigen.

Neue Begriffe in diesem Kapitel

Objekt, Klasse, Instanz, Methode, Signatur, Parameter, Typ, Zustand, Quelltext, Ergebniswert, Compiler

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Zusammenfassung der Konzepte

- **Objekt** Java-Objekte modellieren Objekte eines Anwendungsbereichs.
- **Klasse** Objekte werden durch Klassen erzeugt. Eine Klasse beschreibt eine bestimmte Art von Objekten; Objekte repräsentieren individuelle Instanzen einer Klasse.
- **Methode** Wir können mit Objekten kommunizieren, indem wir ihre Methoden aufrufen. Ein Objekt tut üblicherweise etwas, wenn eine seiner Methoden aufgerufen wird.
- **Parameter** Methoden können Parameter haben, mit denen zusätzliche Informationen für eine Aufgabe angegeben werden.
- **Signatur** Der Name und die Parametertypen einer Methode werden als deren Signatur bezeichnet. Sie benennt die benötigten Informationen für einen Aufruf der Methode.
- **Typ** Parameter haben Typen. Ein Typ definiert, welche Arten von Werten ein Parameter annehmen kann.
- **Eine Klasse, viele Instanzen** Von einer Klasse können viele gleichartige Instanzen erzeugt werden.
- **Zustand** Objekte haben einen Zustand. Dieser Zustand wird durch Werte repräsentiert, die in Datenfeldern gehalten werden.
- **Methodenaufrufe** Objekte können miteinander kommunizieren, indem sie gegenseitig ihre Methoden aufrufen.
- **Quelltext** Der Quelltext einer Klasse legt die Struktur und das Verhalten (die Datenfelder und die Methoden) aller Instanzen dieser Klasse fest.
- **Ergebnis** Methoden können Informationen über ein Objekt durch einen Ergebniswert zurückliefern.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Übung 1.30 In diesem Kapitel haben wir die Datentypen `int` und `String` erwähnt. Java hat einige weitere vordefinierte Datentypen. Finden Sie heraus, welche das sind und welchem Zweck sie dienen. Sie können dies in Anhang B nachlesen oder in einem anderen Buch über Java oder in einem Onlinehandbuch zu Java. Ein englisches Onlinehandbuch ist beispielsweise zu finden unter:

http://java.sun.com/docs/books/tutorial/java/nutsandbolts/datatypes.html

Übung 1.31 Welche Typen haben die folgenden Werte?

```
0
"hallo"
101
-1
true
"33"
3.1415
```

Übung 1.32 Was müssten Sie tun, um einem Kreis-Objekt ein neues Datenfeld, beispielsweise `name`, hinzuzufügen?

Übung 1.33 Schreiben Sie die Signatur für eine Methode `senden`, die einen Parameter vom Typ `String` bekommt und keinen Wert zurückliefert.

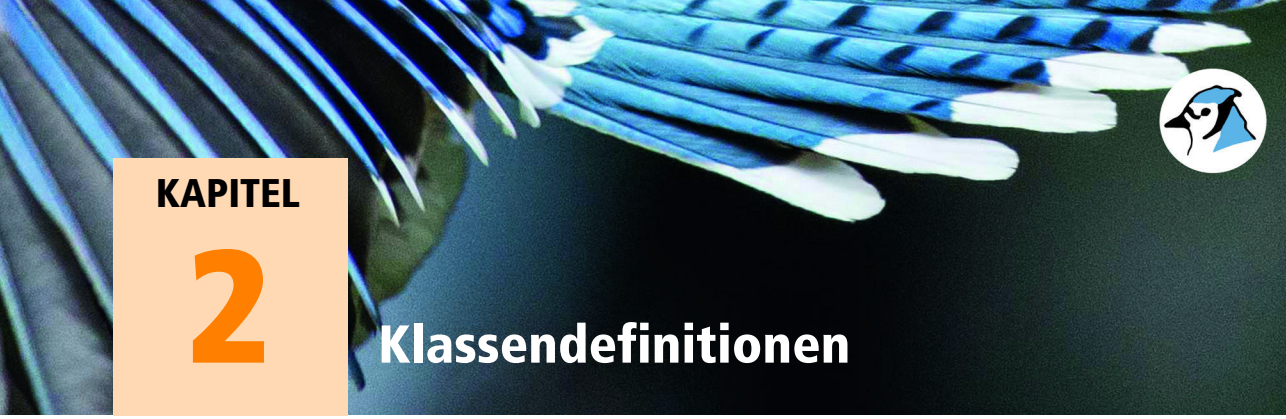
Übung 1.34 Schreiben Sie den Kopf für eine Methode `mittelwert`, die zwei Parameter vom Typ `int` bekommt und einen `int`-Wert zurückliefert.

Übung 1.35 Sehen Sie sich das Buch an, das Sie gerade lesen. Ist es ein Objekt oder eine Klasse? Wenn es eine Klasse ist, dann nennen Sie einige Objekte. Wenn es ein Objekt ist, dann nennen Sie seine Klasse.

Übung 1.36 Kann ein Objekt verschiedene Klassen haben? Diskutieren Sie.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr



KAPITEL 2 Klassendefinitionen

Lernziele

Zentrale Konzepte in diesem Kapitel: Datenfelder, sondierende und verändernde Methoden, Konstruktoren, Zuweisung und bedingte Anweisung, Parameter

Java-Konstrukte in diesem Kapitel: Datenfeld, Konstruktor, Kommentar, Parameter, Zuweisung (`=`), Block, `return`, `void`, zusammengesetzter Zuweisungsoperator (`+=`, `-=`), `if`

In diesem Kapitel befassen wir uns erstmals gründlich mit dem Quelltext einer Klasse. Wir werden die Basiselemente von Klassendefinitionen diskutieren: *Datenfelder*, *Konstruktoren* und *Methoden*. Methoden enthalten Anweisungen. Wir werden zuerst Methoden betrachten, die lediglich einfache arithmetische Anweisungen und Ausgabeanweisungen enthalten. Später führen wir *bedingte Anweisungen* ein, mit denen eine Auswahl zwischen verschiedenen Aktionen innerhalb einer Methode getroffen werden kann.

Wir beginnen, indem wir uns detailliert ein neues Projekt ansehen. Dieses Projekt enthält eine naive Implementierung eines Ticketautomaten. Während wir die elementaren Konzepte von Klassen einführen, werden wir schnell merken, dass diese Implementierung einige Schwächen hat. Entsprechend werden wir anschließend eine weiterentwickelte Version des Ticketautomaten beschreiben, die etliche Verbesserungen aufweist. Schließlich, um die Konzepte dieses Kapitels noch einmal zu wiederholen, werfen wir einen Blick auf die interne Struktur der *Laborkurse* aus dem letzten Kapitel.

2.1 Ticketautomaten

Auf Bahnhöfen und U-Bahnstationen stehen üblicherweise Ticketautomaten, die ein Ticket ausdrucken, sobald ein Kunde den korrekten Betrag eingeworfen hat. In diesem Kapitel werden wir eine Klasse definieren, die einen solchen Automaten modelliert. Da wir hier erstmalig die Struktur einer Klasse untersuchen, ist die Simulation selbst zu Anfang absichtlich simpel gehalten. Auf diese Weise können wir untersuchen, inwieweit solche Modelle von ihren Originalen in der Realität abwei-

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:17 Uhr